In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

In [ ]:
```
import patsy
import hddm
print(hddm.__version__)
```

In [ ]:
```
data = hddm.load_csv('joinhddm_alltimepoints_23March2026.csv')
data.head(10)
```

In [ ]:
```
data = hddm.utils.flip_errors(data)
```

In [ ]:
```
# Standard Global Z-scoring
data['rew_z'] = (data['rew'] - data['rew'].mean()) / data['rew'].std()
data['eff_z'] = (data['eff'] - data['eff'].mean()) / data['eff'].std()
```

In [ ]:
```
data['DBS_state'] = data['DBS_state'].replace({'Pre-DBS':'0_PreDBS', 'OFF':'1_OFF', 'ON': '2_ON'})
```

In [ ]:
```
data['DBS_location'] = data['DBS_location'].replace({'STN':'0_STN', 'GPi':'1_GPi'})
```

Models - simplest model

In [ ]:
```
import hddm
from joblib import Parallel, delayed

# The core formula
formula_refined = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
    "DBS_location:DBS_state:eff_z + "                 # The 6 effort slopes
    "DBS_location:DBS_state:rew_z"                    # The 6 reward slopes
)

def run_middleweight_model_1(id):
    # 'data' must be defined in your environment
    mD1_gorF = hddm.HDDMRegressor(data,
```

```
                                    formula_refined,
                                    group_only_regressors=False,
                                    p_outlier=0.05,
                                    bias=False)           # <--- Explicitly False for Model 1

    mD1_gorF.find_starting_values()

    # MIDDLEWEIGHT RUN: 5,000 samples, 1,500 burn-in
    mD1_gorF.sample(5000, burn=1500, dbname=f'middle_chain_mD1_gorF_{id}.db', db='pickle')
    return mD1_gorF

# Run 4 chains in parallel
print("Starting Middleweight Model 1 Run (Estimating individual slopes)...")
middleweight_mD1_models = Parallel(n_jobs=4)(delayed(run_middleweight_model_1)(i) for i in range(4))

# ======================================================
# SAVING THE MIDDLEWEIGHT ARCHITECTURE
# ======================================================
print("Saving Middleweight Model 1..")
for i, model in enumerate(middleweight_mD1_models):
    save_name = f'middle_mD1_gorF_chain_{i}'
    model.save(save_name)
print("Middleweight Model 1 successfully saved!")
```

Models - threshold (a)

In [ ]:

In [ ]:
```
import hddm
from joblib import Parallel, delayed

formula_refined = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
    "DBS_location:DBS_state:eff_z + "                 # The 6 effort slopes
    "DBS_location:DBS_state:rew_z"                    # The 6 reward slopes
)

def run_refined_model_2(id):
    # 'data' must be defined in your environment
    mD2 = hddm.HDDMRegressor(data,
```

```
                                formula_refined,
                                depends_on={'a': ['DBS_location', 'DBS_state']}, # <--- Added threshold dependency
                                group_only_regressors=False,
                                p_outlier=0.05)

    mD2.find_starting_values()

    # Updated dbname to mD2 so you don't overwrite Model 1 chains
    mD2.sample(5000, burn=1500, dbname=f'refined_chain_mD2_{id}.db', db='pickle')
    return mD2

# Run 4 chains in parallel
refined_interactionD2_models = Parallel(n_jobs=4)(delayed(run_refined_model_2)(i) for i in range(4))

# Run this immediately after your Parallel execution finishes

print("Saving models...")
for i, model in enumerate(refined_interactionD2_models):
    # This creates a file (e.g., 'mD2_model_chain_0') containing the model structure.
    # Note: Do not add a file extension (like .pickle or .db) to this save name.
    save_name = f'mD2_model_chain_{i}'
    model.save(save_name)
    print(f"Successfully saved model architecture for chain {i} as '{save_name}'")

print("All chains saved!")
```

Threshold + bias

In [ ]:

```
import hddm
from joblib import Parallel, delayed

formula_refined = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
    "DBS_location:DBS_state:eff_z + "                 # The 6 effort slopes
    "DBS_location:DBS_state:rew_z"                    # The 6 reward slopes
)

def run_refined_model_3(id):
    # 'data' must be defined in your environment
    mD3 = hddm.HDDMRegressor(data,
```

```
                                formula_refined,
                                depends_on={'a': ['DBS_location', 'DBS_state']}, # <--- Added threshold dependency
                                group_only_regressors=False,
                                p_outlier=0.05,
                               bias=True) #true

    mD3.find_starting_values()

    # Updated dbname to mD3 so you don't overwrite Model 2 chains
    mD3.sample(5000, burn=1500, dbname=f'refined_chain_mD3_{id}.db', db='pickle')
    return mD3

# Run 4 chains in parallel
refined_interactionD3_models = Parallel(n_jobs=4)(delayed(run_refined_model_3)(i) for i in range(4))
```

Threshold + bias + rew*eff interaction

In [ ]:

```
Import hddm
from joblib import Parallel, delayed

formula_4way = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
    "DBS_location:DBS_state:eff_z + "                 # The 6 effort slopes
    "DBS_location:DBS_state:rew_z + "                 # The 6 reward slopes
    "DBS_location:DBS_state:eff_z:rew_z"              # NEW: The 6 effort*reward 4-way interaction slopes
)

def run_refined_model_4(id):
    # 'data' must be defined in your environment
    mD4 = hddm.HDDMRegressor(data,
                             formula_4way,
                             depends_on={'a': ['DBS_location', 'DBS_state']},
                             group_only_regressors=False,
                             p_outlier=0.05,
                             bias=True)

    mD4.find_starting_values()

    # Updated dbname to mD4 so you don't overwrite previous chains
    mD4.sample(5000, burn=1500, dbname=f'refined_chain_mD4_{id}.db', db='pickle')
```

```
    return mD4

# Run 4 chains in parallel
refined_interactionD4_models = Parallel(n_jobs=4)(delayed(run_refined_model_4)(i) for i in range(4))

# =========================================================
# SAVING THE MODEL ARCHITECTURES
# =========================================================
print("Saving Model 4...")
for i, model in enumerate(refined_interactionD4_models):
    save_name = f'mD4_model_chain_{i}'
    model.save(save_name)
print("Model 4 successfully saved!")
```

In [ ]:

In [ ]:

Non-decision time

In [ ]:
```
import os

# =========================================================
# SUPERCOMPUTER FIX: PREVENT THREAD EXPLOSION
# Must be set BEFORE importing hddm
# =========================================================
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
os.environ["NUMEXPR_NUM_THREADS"] = "1"

import hddm
from joblib import Parallel, delayed


# The core formula remains identical
formula_refined = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
```

```
    "DBS_location:DBS_state:eff_z + "                  # The 6 effort slopes
    "DBS_location:DBS_state:rew_z"                     # The 6 reward slopes
)

def run_middleweight_model_2T(id):
    # 'data' must be defined in your environment
    mD2T_gorF = hddm.HDDMRegressor(data,
                                   formula_refined,
                                   depends_on={'t': ['DBS_location', 'DBS_state']},
                                   group_only_regressors=False,
                                   p_outlier=0.05)


    mD2T_gorF.find_starting_values()

    # MIDDLEWEIGHT RUN: 5,000 samples, 1,500 burn-in
    mD2T_gorF.sample(5000, burn=1500, dbname=f'middle_chain_mD2T_gorF_{id}.db', db='pickle')
    return mD2T_gorF

# Run 4 chains in parallel
print("Starting Middleweight Model 2T Run (Estimating individual slopes)...")
middleweight_mD2T_models = Parallel(n_jobs=4)(delayed(run_middleweight_model_2T)(i) for i in range(4))

# ========================================================
# SAVING THE MIDDLEWEIGHT ARCHITECTURE
# ========================================================
print("Saving Middleweight Model 2T...")
for i, model in enumerate(middleweight_mD2T_models):
    save_name = f'middle_mD2T_gorF_chain_{i}'
    model.save(save_name)
print("Middleweight Model 2T successfully saved!")
```

Non decision time + bias

In [ ]:

```
import os

# ========================================================
# SUPERCOMPUTER FIX: PREVENT THREAD EXPLOSION
# Must be set BEFORE importing hddm
# ========================================================
```

```
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
os.environ["NUMEXPR_NUM_THREADS"] = "1"

import hddm
from joblib import Parallel, delayed


# The core formula remains identical
formula_refined = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
    "DBS_location:DBS_state:eff_z + "                 # The 6 effort slopes
    "DBS_location:DBS_state:rew_z"                    # The 6 reward slopes
)

def run_middleweight_model_2Tb(id):
    # 'data' must be defined in your environment
    mD2Tb_gorF = hddm.HDDMRegressor(data,
                                    formula_refined,
                                    depends_on={'t': ['DBS_location', 'DBS_state']},
                                    group_only_regressors=False,
                                    p_outlier=0.05,
                                    bias=True) # <--- Explicitly True for Model 2Tb

    mD2Tb_gorF.find_starting_values()

    # MIDDLEWEIGHT RUN: 5,000 samples, 1,500 burn-in
    mD2Tb_gorF.sample(5000, burn=1500, dbname=f'middle_chain_mD2Tb_gorF_{id}.db', db='pickle')
    return mD2Tb_gorF

# Run 4 chains in parallel
print("Starting Middleweight Model 2Tb Run (Estimating individual slopes)...")
middleweight_mD2Tb_models = Parallel(n_jobs=4)(delayed(run_middleweight_model_2Tb)(i) for i in range(4))

# ======================================================
# SAVING THE MIDDLEWEIGHT ARCHITECTURE
# ======================================================
print("Saving Middleweight Model 2Tb...")
for i, model in enumerate(middleweight_mD2Tb_models):
```

```
    save_name = f'middle_mD2Tb_gorF_chain_{i}'
    model.save(save_name)
print("Middleweight Model 2Tb successfully saved!")
```

Non decision time + bias + rew*eff

In [ ]:

```
import os

# ========================================================
# SUPERCOMPUTER FIX: PREVENT THREAD EXPLOSION
# Must be set BEFORE importing hddm
# ========================================================
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
os.environ["NUMEXPR_NUM_THREADS"] = "1"

import hddm
from joblib import Parallel, delayed

# The 4-way interaction formula
formula_4way = (
    "v ~ 0 + DBS_location:DBS_state + "               # The intercepts
    "DBS_location:DBS_state:eff_z + "                 # The effort slopes
    "DBS_location:DBS_state:rew_z + "                 # The reward slopes
    "DBS_location:DBS_state:eff_z:rew_z"              # The effort x reward interaction
)

def run_middleweight_model_4T(id):
    # 'data' must be defined in your environment
    mD4T_gorF = hddm.HDDMRegressor(data,
                                   formula_4way,
                                   depends_on={'t': ['DBS_location', 'DBS_state']},
                                   group_only_regressors=False,
                                   p_outlier=0.05,
                                   bias=True)

    mD4T_gorF.find_starting_values()
```

```
    # MIDDLEWEIGHT RUN: 5,000 samples, 1,500 burn-in
    mD4T_gorF.sample(5000, burn=1500, dbname=f'middle_chain_mD4T_gorF_{id}.db', db='pickle')
    return mD4T_gorF

# Run 4 chains in parallel
print("Starting Middleweight Model 4T Run (Testing 4-way interaction)...")
middleweight_mD4T_models = Parallel(n_jobs=4)(delayed(run_middleweight_model_4T)(i) for i in range(4))

# ======================================================
# SAVING THE MIDDLEWEIGHT ARCHITECTURE
# ======================================================
print("Saving Middleweight Model 4T...")
for i, model in enumerate(middleweight_mD4T_models):
    save_name = f'middle_mD4T_gorF_chain_{i}'
    model.save(save_name)
print("Middleweight Model 4T successfully saved!")
```

Model with both a and t varying per group per state

In [ ]:

```
import os

# ======================================================
# SUPERCOMPUTER FIX: PREVENT THREAD EXPLOSION
# Must be set BEFORE importing hddm
# ======================================================
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
os.environ["NUMEXPR_NUM_THREADS"] = "1"

import hddm
from joblib import Parallel, delayed

# The core formula remains identical
formula_refined = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
    "DBS_location:DBS_state:eff_z + "                 # The 6 effort slopes
    "DBS_location:DBS_state:rew_z"                    # The 6 reward slopes
)
```

```
def run_middleweight_model_6(id):
    # 'data' must be defined in your environment
    mD6_gorF = hddm.HDDMRegressor(data,
                                  formula_refined,
                                  depends_on={'t': ['DBS_location', 'DBS_state'], 'a': ['DBS_location', 'DBS_state']},
                                  group_only_regressors=False,
                                  p_outlier=0.05,
                                  bias=True) # <--- Explicitly True for Model 6

    mD6_gorF.find_starting_values()

    # MIDDLEWEIGHT RUN: 5,000 samples, 1,500 burn-in
    mD6_gorF.sample(5000, burn=1500, dbname=f'middle_chain_mD6_gorF_{id}.db', db='pickle')
    return mD6_gorF

# Run 4 chains in parallel
print("Starting Middleweight Model 6 Run (Estimating individual slopes)...")
middleweight_mD6_models = Parallel(n_jobs=4)(delayed(run_middleweight_model_6)(i) for i in range(4))

# ======================================================
# SAVING THE MIDDLEWEIGHT ARCHITECTURE
# ======================================================
print("Saving Middleweight Model 6...")
for i, model in enumerate(middleweight_mD6_models):
    save_name = f'middle_mD6_gorF_chain_{i}'
    model.save(save_name)
print("Middleweight Model 6 successfully saved!")
```

Model comparisons- change names of models in code below, depending which models you are comparing

In [ ]:

```
import kabuki
import numpy as np
from pymc.diagnostics import effective_n

def evaluate_hddm_models(model_dict, rhat_cutoff=1.02, ess_cutoff=400):
    """
    Evaluates DIC, Gelman-Rubin (R-hat), and Effective Sample Size (ESS).
    model_dict: Dictionary where keys are model names, and values are lists of chain objects.
```

```
"""
print("====== STRICT MODEL EVALUATION ======\n")

# Dictionary to store the DIC scores for the final comparison
dic_scores = {}

for name, chains in model_dict.items():
    print(f"--- {name} ---")

    # 1. Evaluate DIC
    dic_mean = np.mean([m.dic for m in chains])
    dic_scores[name] = dic_mean  # Save for later
    print(f"DIC: {dic_mean:.2f}")

    # 2. Evaluate R-hat
    rhats = kabuki.analyze.gelman_rubin(chains)
    bad_rhats = {param: val for param, val in rhats.items() if val > rhat_cutoff}

    if bad_rhats:
        print(f"R-hat WARNING: {len(bad_rhats)} parameters > {rhat_cutoff}")
        for param, val in list(bad_rhats.items())[:5]:
            print(f"  {param}: {val:.4f}")
        if len(bad_rhats) > 5: print("  ...and more.")
    else:
        print(f"R-hat: PASS (All parameters < {rhat_cutoff})")

    # 3. Evaluate ESS
    bad_ess = {}
    for param in rhats.keys():
        try:
            trace_matrix = np.array([m.nodes_db.node[param].trace() for m in chains])
            ess = effective_n(trace_matrix)
            ess_value = float(np.squeeze(ess))

            if ess_value < ess_cutoff:
                bad_ess[param] = ess_value
        except KeyError:
            continue

    if bad_ess:
        print(f"ESS WARNING: {len(bad_ess)} parameters < {ess_cutoff}")
```

```
            for param, val in list(bad_ess.items())[:5]:
                print(f"  {param}: ESS = {val:.1f}")
            if len(bad_ess) > 5: print("  ...and more.")
        else:
            print(f"ESS: PASS (All parameters > {ess_cutoff})")

        print("\n")

    # ========================================================
    # FINAL DIC COMPARISON TABLE
    # ========================================================
    print("====== FINAL DIC COMPARISON SUMMARY ======")

    # Sort models by DIC (Lowest is best)
    sorted_models = sorted(dic_scores.items(), key=lambda x: x[1])
    best_dic = sorted_models[0][1]

    for i, (name, dic_val) in enumerate(sorted_models):
        diff = dic_val - best_dic
        if i == 0:
            print(f"1. {name}: {dic_val:.2f}  <-- WINNER")
        else:
            print(f"{i+1}. {name}: {dic_val:.2f}  (+{diff:.2f} worse than winner)")
    print("==========================================\n")

# ========================================
# HOW TO RUN IT WITH YOUR NEW DATA:
# ========================================
# Compare the previous winner against the new contender

models_to_compare = {
    "Model 2 (Drift + Threshold)": [model_name], #replace with model name
    "Model 3 (Drift + Threshold + Bias)": [model_name] #replace with model name
}

# Run it!
evaluate_hddm_models(models_to_compare)
```

Winning model - run again - increase sampling and chains (takes ages to run)

In [ ]:

```python
import os

# ========================================================
# SUPERCOMPUTER FIX: PREVENT THREAD EXPLOSION
# Must be set BEFORE importing hddm
# ========================================================
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
os.environ["NUMEXPR_NUM_THREADS"] = "1"

import hddm
from joblib import Parallel, delayed


# The core formula remains identical
formula_refined = (
    "v ~ 0 + DBS_location:DBS_state + "               # The 6 intercepts (baselines)
    "DBS_location:DBS_state:eff_z + "                 # The 6 effort slopes
    "DBS_location:DBS_state:rew_z"                    # The 6 reward slopes
)

def run_massive_model_2TbF(id):

    mD2TbF_gorF = hddm.HDDMRegressor(data,
                                formula_refined,
                                depends_on={'t': ['DBS_location', 'DBS_state']},
                                group_only_regressors=False, #
                                p_outlier=0.05,
                                bias=True) # <--- Explicitly True for Model 2TbF

    mD2TbF_gorF.find_starting_values()

    # Massive RUN: 10 000 samples, 5 000 burn-in
    mD2TbF_gorF.sample(10000, burn=5000, dbname=f'massive_chain_mD2TbF_gorF_{id}.db', db='pickle')
    return mD2TbF_gorF

# Run 8 chains in parallel
print("Starting Massive Model 2TbF Run (Estimating individual slopes)...")
```

```
massive_mD2TbF_models = Parallel(n_jobs=8)(delayed(run_massive_model_2TbF)(i) for i in range(8))

# ========================================================
# SAVING THE Massive ARCHITECTURE
# ========================================================
print("Saving Massive Model 2TbF...")
for i, model in enumerate(massive_mD2TbF_models):
    save_name = f'massive_mD2TbF_gorF_chain_{i}'
    model.save(save_name)
print("Massive Model 2TbF successfully saved!")
```

Now skip to model diagnostics & PPC section, before doing hypothesis testing

Hypothesis testing - load final model

In [ ]:
```
#mD2TbF - this has 10 000 samples (Final)


loaded_mD2TbF_models = []
num_chains = 8

print("Loading Massive Model 2TbF chains...")

# Loop through and load each saved file
for i in range(num_chains):
    load_name = f'massive_mD2TbF_gorF_chain_{i}'

    # hddm.load() reads the saved file and reconstructs the model object
    model = hddm.load(load_name)
    loaded_mD2TbF_models.append(model)

    print(f"Successfully loaded {load_name}")

print("\nAll Middleweight Model 2TbF chains loaded successfully!")
```

In [ ]:
```
Hypothesis testing - Pre vs OFF, OFF vs ON
```

In [ ]:
```
import pandas as pd
import numpy as np
```

```python
# 1. EXTRACT AND COMBINE ALL TRACES
print("Extracting posterior traces from all chains...")
# Assuming loaded_mD2T_models is a list of your loaded model objects
traces = pd.concat([model.get_traces() for model in loaded_mD2TbF_models], ignore_index=True)

# ==============================================================================
# 2. EXACT COLUMN MAPPING (STN and GPi)
# ==============================================================================
nodes = {
    # Baseline (Intercepts)
    'v_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]',
    'v_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]',
    'v_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]',
    'v_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]',
    'v_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]',
    'v_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]',

    # Effort Slopes
    'eff_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]:eff_z',
    'eff_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]:eff_z',
    'eff_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]:eff_z',
    'eff_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]:eff_z',
    'eff_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]:eff_z',
    'eff_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]:eff_z',

    # Reward Slopes
    'rew_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]:rew_z',
    'rew_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]:rew_z',
    'rew_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]:rew_z',
    'rew_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]:rew_z',
    'rew_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]:rew_z',
    'rew_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]:rew_z',

    # Non-decision times (Assuming these are actually non-decision times 't' based on the name)
    't_STN_Pre': 't(0_STN.0_PreDBS)',
    't_GPi_Pre': 't(1_GPi.0_PreDBS)',
    't_STN_OFF': 't(0_STN.1_OFF)',
    't_GPi_OFF': 't(1_GPi.1_OFF)',
    't_STN_ON':  't(0_STN.2_ON)',
    't_GPi_ON':  't(1_GPi.2_ON)'
```

```
}

# ==============================================================================
# 3. HYPOTHESIS TESTING ENGINE
# ==============================================================================
results_list = []

def test_hypothesis(test_name, param_type, diff_trace):
    """Calculates Bayesian stats from the subtracted traces."""
    mean_diff = diff_trace.mean()
    cri_lower = np.percentile(diff_trace, 2.5)
    cri_upper = np.percentile(diff_trace, 97.5)

    # Probability of Direction (What % of the trace is strictly greater than 0?)
    p_dir = (diff_trace > 0).mean()

    # In Bayesian stats, if P >= 0.975 or P <= 0.025, it excludes zero at a 95% confidence level.
    is_sig = "Yes" if (p_dir >= 0.975 or p_dir <= 0.025) else "No"

    # Determine Bayesian Evidence Level
    # We take the max of p_dir or (1 - p_dir) to measure the strength of the effect in *either* direction
    p_max = max(p_dir, 1 - p_dir)

    if p_max >= 0.99:
        evidence = "Very Strong"
    elif p_max >= 0.975:
        evidence = "Strong"
    elif p_max >= 0.95:
        evidence = "Moderate"
    elif p_max >= 0.90:
        evidence = "Weak"
    else:
        evidence = "None"

    results_list.append({
        "Hypothesis Test": test_name,
        "Parameter": param_type,
        "Mean Difference": round(mean_diff, 4),
        "95% CrI Lower": round(cri_lower, 4),
        "95% CrI Upper": round(cri_upper, 4),
        "P(Diff > 0)": round(p_dir, 4),
```

```
        "Significant? (95%)": is_sig,
        "Evidence Level": evidence
    })

# Define the 4 parameter types we are testing
param_types = [('v', 'Baseline Drift'), ('eff', 'Effort Slope'), ('rew', 'Reward Slope'), ('t', 'Non-decision time')]

print("Running Bayesian Hypothesis Tests...")

for p_prefix, p_label in param_types:

    # --- QUESTION 1: Group differences at PreDBS ---
    diff_Q1 = traces[nodes[f'{p_prefix}_STN_Pre']] - traces[nodes[f'{p_prefix}_GPi_Pre']]
    test_hypothesis("Q1: Group Diff at PreDBS (STN - GPi)", p_label, diff_Q1)

    # --- QUESTION 2: PreDBS to OFF ---
    # Within STN
    diff_Q2_STN = traces[nodes[f'{p_prefix}_STN_OFF']] - traces[nodes[f'{p_prefix}_STN_Pre']]
    test_hypothesis("Q2: Within STN (OFF - PreDBS)", p_label, diff_Q2_STN)
    # Within GPi
    diff_Q2_GPi = traces[nodes[f'{p_prefix}_GPi_OFF']] - traces[nodes[f'{p_prefix}_GPi_Pre']]
    test_hypothesis("Q2: Within GPi (OFF - PreDBS)", p_label, diff_Q2_GPi)
    # Across Groups (Difference of Differences)
    diff_of_diff_Q2 = diff_Q2_STN - diff_Q2_GPi
    test_hypothesis("Q2: Diff-of-Diff Across Groups (OFF - PreDBS)", p_label, diff_of_diff_Q2)

    # --- QUESTION 3: OFF to ON ---
    # Within STN
    diff_Q3_STN = traces[nodes[f'{p_prefix}_STN_ON']] - traces[nodes[f'{p_prefix}_STN_OFF']]
    test_hypothesis("Q3: Within STN (ON - OFF)", p_label, diff_Q3_STN)
    # Within GPi
    diff_Q3_GPi = traces[nodes[f'{p_prefix}_GPi_ON']] - traces[nodes[f'{p_prefix}_GPi_OFF']]
    test_hypothesis("Q3: Within GPi (ON - OFF)", p_label, diff_Q3_GPi)
    # Across Groups (Difference of Differences)
    diff_of_diff_Q3 = diff_Q3_STN - diff_Q3_GPi
    test_hypothesis("Q3: Diff-of-Diff Across Groups (ON - OFF)", p_label, diff_of_diff_Q3)

# ==============================================================================
# 4. EXPORT TO CSV
# ==============================================================================
results_df = pd.DataFrame(results_list)
```

```
results_df.to_csv("HDDM_Hypothesis_Testing_Results_temp_mD2TbF_29May2026.csv", index=False)
print("\nSuccess! Results exported to 'HDDM_Hypothesis_Testing_Results_temp_mD2TbF.csv'")

# If you are in a Jupyter Notebook, this will display the first 10 rows nicely
try:
    display(results_df.head(10))
except NameError:
    print(results_df.head(10))
```

In [ ]:

Hypothesis testing - Pre-DBS vs ON

In [ ]:
```
import pandas as pd
import numpy as np

# 1. EXTRACT AND COMBINE ALL TRACES
print("Extracting posterior traces from all chains...")
# Assuming loaded_mD2TbF_models is a list of your loaded model objects
traces = pd.concat([model.get_traces() for model in loaded_mD2TbF_models], ignore_index=True)

# ==============================================================================
# 2. EXACT COLUMN MAPPING (STN and GPi)
# ==============================================================================
nodes = {
    # Baseline Speed (Intercepts)
    'v_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]',
    'v_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]',
    'v_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]',
    'v_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]',
    'v_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]',
    'v_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]',

    # Effort Slopes
    'eff_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]:eff_z',
    'eff_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]:eff_z',
    'eff_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]:eff_z',
    'eff_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]:eff_z',
    'eff_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]:eff_z',
    'eff_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]:eff_z',
```

```
    # Reward Slopes
    'rew_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]:rew_z',
    'rew_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]:rew_z',
    'rew_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]:rew_z',
    'rew_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]:rew_z',
    'rew_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]:rew_z',
    'rew_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]:rew_z',

    #  Non-decision times
    't_STN_Pre': 't(0_STN.0_PreDBS)',
    't_GPi_Pre': 't(1_GPi.0_PreDBS)',
    't_STN_OFF': 't(0_STN.1_OFF)',
    't_GPi_OFF': 't(1_GPi.1_OFF)',
    't_STN_ON':  't(0_STN.2_ON)',
    't_GPi_ON':  't(1_GPi.2_ON)'
}

# ==============================================================================
# 3. HYPOTHESIS TESTING ENGINE
# ==============================================================================
results_list = []

def test_hypothesis(test_name, param_type, diff_trace):
    """Calculates Bayesian stats from the subtracted traces."""
    mean_diff = diff_trace.mean()
    cri_lower = np.percentile(diff_trace, 2.5)
    cri_upper = np.percentile(diff_trace, 97.5)

    # Probability of Direction (What % of the trace is strictly greater than 0?)
    p_dir = (diff_trace > 0).mean()

    # In Bayesian stats, if P >= 0.975 or P <= 0.025, it excludes zero at a 95% confidence level.
    is_sig = "Yes" if (p_dir >= 0.975 or p_dir <= 0.025) else "No"

    # Determine Bayesian Evidence Level
    p_max = max(p_dir, 1 - p_dir)

    if p_max >= 0.99:
        evidence = "Very Strong"
    elif p_max >= 0.975:
```

```
        evidence = "Strong"
    elif p_max >= 0.95:
        evidence = "Moderate"
    elif p_max >= 0.90:
        evidence = "Weak"
    else:
        evidence = "None"

    results_list.append({
        "Hypothesis Test": test_name,
        "Parameter": param_type,
        "Mean Difference": round(mean_diff, 4),
        "95% CrI Lower": round(cri_lower, 4),
        "95% CrI Upper": round(cri_upper, 4),
        "P(Diff > 0)": round(p_dir, 4),
        "Significant? (95%)": is_sig,
        "Evidence Level": evidence
    })

# Define the 4 parameter types we are testing
param_types = [('v', 'Baseline Drift'), ('eff', 'Effort Slope'), ('rew', 'Reward Slope'), ('t', 'Non-decision time')]

print("Running Bayesian Hypothesis Tests...")

for p_prefix, p_label in param_types:

    # --- QUESTION 1: Group differences at PreDBS ---
    diff_Q1 = traces[nodes[f'{p_prefix}_STN_Pre']] - traces[nodes[f'{p_prefix}_GPi_Pre']]
    test_hypothesis("Q1: Group Diff at PreDBS (STN - GPi)", p_label, diff_Q1)

    # --- QUESTION 2: PreDBS to ON ---
    # Within STN
    diff_Q2_STN = traces[nodes[f'{p_prefix}_STN_ON']] - traces[nodes[f'{p_prefix}_STN_Pre']]
    test_hypothesis("Q2: Within STN (ON - PreDBS)", p_label, diff_Q2_STN)

    # Within GPi
    diff_Q2_GPi = traces[nodes[f'{p_prefix}_GPi_ON']] - traces[nodes[f'{p_prefix}_GPi_Pre']]
    test_hypothesis("Q2: Within GPi (ON - PreDBS)", p_label, diff_Q2_GPi)

    # Across Groups (Difference of Differences)
    diff_of_diff_Q2 = diff_Q2_STN - diff_Q2_GPi
```

```
    test_hypothesis("Q2: Diff-of-Diff Across Groups (ON - PreDBS)", p_label, diff_of_diff_Q2)

# ================================================================================
# 4. EXPORT TO CSV
# ================================================================================
results_df = pd.DataFrame(results_list)
results_df.to_csv("HDDM_Hypothesis_Testing_Results_temp_mD2TbF_Pre_vs_ON_29May2026.csv", index=False)
print("\nSuccess! Results exported to 'HDDM_Hypothesis_Testing_Results_temp_mD2TbF_Pre_vs_ON.csv'")

# If you are in a Jupyter Notebook, this will display the first 10 rows nicely
try:
    display(results_df.head(10))
except NameError:
    print(results_df.head(10))
```

In [ ]:
```
Posterior predictive checks
```

In [ ]:
```
import hddm
import pandas as pd
import numpy as np

# --- 2. GENERATE PPC ---
ppc_frames = []

for i, model in enumerate(loaded_mD2TbF_models):
    print(f"Generating PPC for chain {i}...")

    # CRITICAL CHANGE: Turn off append_data.
    # This prevents HDDM from trying (and failing) to map indices.
    ppc_chain = hddm.utils.post_pred_gen(model, samples=10, append_data=False)

    # Reset the index so 'node' becomes a regular column we can manipulate
    ppc_chain = ppc_chain.reset_index()
    ppc_chain['chain_id'] = i

    # MANUALLY EXTRACT CONDITIONS FROM THE NODE STRING
    # The node looks like: "wfpt(1_GPi.2_ON).1500"
    # 1. Extract everything inside the parentheses: "1_GPi.2_ON"
    condition_string = ppc_chain['node'].str.extract(r'\((.*?)\)')[0]
```

```
    # 2. Split the string by the period into two new columns
    split_conds = condition_string.str.split('.', expand=True)
    ppc_chain['DBS_location'] = split_conds[0]
    ppc_chain['DBS_state'] = split_conds[1]

    ppc_frames.append(ppc_chain)

# Concatenate all chains
ppc_data12 = pd.concat(ppc_frames, ignore_index=True)
print("\nPPC generation complete!")


# --- 3. AGGREGATE OBSERVED DATA ---
conditions = ['DBS_location', 'DBS_state']

obs_stats = data.groupby(conditions).agg(
    obs_accuracy=('response', 'mean'),
    obs_rt=('rt', lambda x: x.abs().mean())
).reset_index()


# --- 4. EXTRACT AND AGGREGATE SIMULATED DATA ---
# When append_data=False, HDDM sometimes names the simulated column 'rt' instead of 'rt_sampled'.
# This line dynamically checks which one it used.
sim_col = 'rt_sampled' if 'rt_sampled' in ppc_data12.columns else 'rt'

ppc_data12['sim_choice_extracted'] = (ppc_data12[sim_col] > 0).astype(int)
ppc_data12['sim_rt_extracted'] = ppc_data12[sim_col].abs()

sim_stats = ppc_data12.groupby(conditions).agg(
    sim_accuracy=('sim_choice_extracted', 'mean'),
    sim_rt=('sim_rt_extracted', 'mean')
).reset_index()


# --- 5. MERGE & COMPARE ---
ppc_comparison = obs_stats.merge(sim_stats, on=conditions, how='outer')

with pd.option_context('display.max_rows', None, 'display.max_columns', None):
```

```
    print("\n--- PPC Summary (Observed vs. Simulated) ---")
    print(ppc_comparison)
```

In [ ]:
```
Simulated vs Obs choice
```

In [ ]:
```
mport matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# 1. PREPARE THE DATA
# Subset only the choice accuracy columns
plot_df = ppc_comparison[['DBS_location', 'DBS_state', 'obs_accuracy', 'sim_accuracy']].copy()

# Rename for a cleaner legend
plot_df = plot_df.rename(columns={'obs_accuracy': 'Observed', 'sim_accuracy': 'Simulated'})

# Melt the dataframe into a "long" format required by Seaborn for grouped bars
melted_df = plot_df.melt(
    id_vars=['DBS_location', 'DBS_state'],
    value_vars=['Observed', 'Simulated'],
    var_name='Data Type',
    value_name='Proportion Acceptance'
)

# 2. SET UP PLOT AESTHETICS
sns.set_theme(style="whitegrid", context="paper", font_scale=1.2)

# Updated colors: Grey for Observed, Scientific Blue for Simulated
color_palette = {'Observed': '#7f8c8d', 'Simulated': '#2980b9'}

# 3. CREATE THE FACETED BAR PLOT
g = sns.catplot(
    data=melted_df,
    x='DBS_state',
    y='Proportion Acceptance',
    hue='Data Type',
    col='DBS_location',
    kind='bar',
    palette=color_palette,
    height=5,
```

```
    aspect=1.2,
    order=['0_PreDBS', '1_OFF', '2_ON'], # Ensures x-axis is chronologically sorted
    legend_out=True
)

# 4. ADD PERCENTAGE LABELS ABOVE BARS (THE FIX)
for ax in g.axes.flat:
    # Iterate through the bar containers (Observed bars, then Simulated bars)
    for container in ax.containers:

        # Calculate and format the labels manually to avoid version conflicts
        labels = [f'{bar.get_height()*100:.1f}%' if bar.get_height() > 0 else '' for bar in container]

        ax.bar_label(
            container,
            labels=labels,      # Pass the pre-calculated string list
            label_type='edge',
            padding=3,
            fontsize=10.5
        )

    # Force the Y-axis to go strictly from 0 to 1
    ax.set_ylim(0, 1.0)

# 5. FINAL FORMATTING
g.set_axis_labels("DBS State", "Proportion Acceptance")
g.set_titles(col_template="{col_name}") # Titles the panels '0_STN' and '1_GPi'

# Save as high-resolution PDF for publication
plt.savefig('PPC_Choice_Barplot29April2026.png', dpi=300, bbox_inches='tight')

# Show the plot
plt.show()
```

Simulated vs Obs RT

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

```python
# 1. EXTRACT RAW RT DATA FOR DISTRIBUTIONS
# Get the raw observed absolute RTs
obs_dist = data[['DBS_location', 'DBS_state', 'rt']].copy()
obs_dist['rt'] = obs_dist['rt'].abs()
obs_dist['Data Type'] = 'Observed'

# Get the raw simulated absolute RTs
# (Using the dynamic sim_col we defined earlier)
sim_col = 'rt_sampled' if 'rt_sampled' in ppc_data12.columns else 'rt'
sim_dist = ppc_data12[['DBS_location', 'DBS_state', sim_col]].copy()
sim_dist = sim_dist.rename(columns={sim_col: 'rt'})
sim_dist['rt'] = sim_dist['rt'].abs()
sim_dist['Data Type'] = 'Simulated'

# Combine into one long dataframe
rt_dist_df = pd.concat([obs_dist, sim_dist], ignore_index=True)

# 2. SET UP PLOT AESTHETICS
sns.set_theme(style="whitegrid", context="paper", font_scale=1.2)
color_palette = {'Observed': '#7f8c8d', 'Simulated': '#2980b9'} # Grey and Blue

# 3. CREATE THE FACETED HISTOGRAMS
# Columns = DBS_location, Rows = DBS_state
g = sns.displot(
    data=rt_dist_df,
    x='rt',
    hue='Data Type',
    col='DBS_location',
    row='DBS_state',
    kind='hist',
    palette=color_palette,
    stat='density',       # Normalizes the y-axis so Obs and Sim are comparable
    common_norm=False,    # Normalizes each distribution independently
    element='step',       # Uses outlines instead of solid bars for better overlap visibility
    binwidth=0.05,        # <--- FIX: Forces bins to be exactly 50ms wide
    binrange=(0, 5.0),    # <--- FIX: Only calculates bins up to 5.0s, dropping the long tail
    alpha=0.6,            # Adds slight transparency
    height=3.5,
    aspect=1.5,
    row_order=['0_PreDBS', '1_OFF', '2_ON']
)
```

```
# 4. FINAL FORMATTING
g.set_axis_labels("Reaction Time (s)", "Density")
g.set_titles(col_template="{col_name}", row_template="{row_name}")

# Limit the x-axis to zoom in on the core distribution
g.set(xlim=(0, 5.0))

# Save as high-resolution PNG for publication
plt.savefig('PPC_RT_Histograms4May2026.png', dpi=300, bbox_inches='tight')

# Show the plot
plt.show()
```

In [ ]:

```
# this is from model mD2TbF

import numpy as np
import pandas as pd
from scipy import stats

# Ensure we are using the absolute RTs
obs = observed_rts.values
sim = simulated_rts.values

# 1. Define the percentiles we want to check
percentiles_to_calc = [1, 5, 10, 25, 50, 75, 90, 95, 99]

# 2. Build a comprehensive list of metrics
metric_names = [
    'Mean RT',
    'SD RT',
    'Skewness',
    'Kurtosis',
    'Min RT',
    'Max RT',
    'IQR (75th - 25th)'
] + [f'{p}th Percentile' for p in percentiles_to_calc]

# Calculate observed metrics
obs_metrics = [
```

```
    np.mean(obs),
    np.std(obs),
    stats.skew(obs),
    stats.kurtosis(obs),
    np.min(obs),
    np.max(obs),
    np.percentile(obs, 75) - np.percentile(obs, 25)
] + [np.percentile(obs, p) for p in percentiles_to_calc]

# Calculate simulated metrics
sim_metrics = [
    np.mean(sim),
    np.std(sim),
    stats.skew(sim),
    stats.kurtosis(sim),
    np.min(sim),
    np.max(sim),
    np.percentile(sim, 75) - np.percentile(sim, 25)
] + [np.percentile(sim, p) for p in percentiles_to_calc]

# 3. Create DataFrame for clean viewing
df_metrics = pd.DataFrame({
    'Metric': metric_names,
    'Observed': obs_metrics,
    'Simulated': sim_metrics
})

# Calculate the absolute difference between Observed and Simulated
df_metrics['Absolute Difference'] = np.abs(df_metrics['Observed'] - df_metrics['Simulated'])

# Optional but helpful: Calculate the percentage difference to give the errors some scale
df_metrics['% Difference'] = (df_metrics['Absolute Difference'] / np.abs(df_metrics['Observed'])) * 100

# 4. Run Kolmogorov-Smirnov Test for overall distribution fit
ks_stat, ks_pvalue = stats.ks_2samp(obs, sim)

# --- PRINT RESULTS ---
print("====== EXTENDED REACTION TIME METRICS COMPARISON (Model: mD2TbF) ======")
# Using a lambda to format floats to 4 decimal places for readability
print(df_metrics.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
print("=======================================================================")
```

```
print(f"Kolmogorov-Smirnov Statistic (D): {ks_stat:.4f}")
print(f"KS p-value: {ks_pvalue:.4e}")
print("================================================================================")
```

Rhat & ESS - run code for each model - change model name in code

In [ ]:

```
import pandas as pd
import numpy as np
from kabuki.analyze import gelman_rubin

# 1. Calculate R-hat as before
r_hat_dict = gelman_rubin(loaded_mD2TbF_models)

# 2. Manual ESS Calculation Function
def get_manual_ess(trace):
    """Calculates Effective Sample Size based on autocorrelation."""
    n = len(trace)
    if n < 2: return 0

    # Calculate autocorrelation
    def autocorr(x, l):
        return np.corrcoef(x[l:], x[:-l])[0, 1] if l < n else 0

    # Sum autocorrelations until they drop below 0.05 (standard heuristic)
    sum_rho = 0
    for l in range(1, min(500, n)):
        rho = autocorr(trace, l)
        if rho < 0.05: break
        sum_rho += rho

    ess = n / (1 + 2 * sum_rho)
    return ess

# 3. Build the combined diagnostic table
results = []
stochastic_nodes = loaded_mD2TbF_models[0].nodes_db[loaded_mD2TbF_models[0].nodes_db['stochastic'] == True].index

print("Calculating ESS for all nodes (this may take a minute)...")

for node in stochastic_nodes:
```

```
    # Get R-hat
    rhat = r_hat_dict.get(node, np.nan)

    # Calculate Total ESS across all 8 chains
    total_ess = 0
    for m in loaded_mD2TbF_models:
        trace = m.nodes_db.node[node].trace()
        total_ess += get_manual_ess(trace)

    results.append({'Parameter': node, 'R-hat': rhat, 'ESS': total_ess})

diagnostic_df = pd.DataFrame(results).set_index('Parameter')

# 4. Filter for struggling parameters
# Nature standard: R-hat > 1.01 OR ESS < 800 (100 per chain)
struggling = diagnostic_df[(diagnostic_df['R-hat'] > 1.01) | (diagnostic_df['ESS'] < 800)]

print("\n====== MCMC CONVERGENCE SUMMARY (8 CHAINS) ======")
print(f"Max R-hat: {diagnostic_df['R-hat'].max():.4f}")
print(f"Min ESS:   {diagnostic_df['ESS'].min():.1f}")
print(f"Mean ESS:  {diagnostic_df['ESS'].mean():.1f}")
print("-" * 50)

if not struggling.empty:
    print(f"Found {len(struggling)} parameters requiring attention (R-hat > 1.01 or ESS < 800):")
    print(struggling.sort_values('R-hat', ascending=False).head(15))
else:
    print("Success: All parameters meet the high-excellence criteria!")
```

Inspect 'hairy catepillars' - run for each model - change model name

In [ ]:

```
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import numpy as np
import os

# 1. Create the output directory if it doesn't exist
output_folder = 'Hairy_chains'
if not os.path.exists(output_folder):
    os.makedirs(output_folder)
```

```
    print(f"Created folder: {output_folder}")

# 2. Identify group nodes (Hyperparameters)
group_nodes = [n for n in loaded_mD2TbF_models[0].nodes_db.query('stochastic == True').index if 'subj' not in n]

# 3. Define the Color Palette
colors = cm.viridis(np.linspace(0, 1, 8))

# 4. Loop through and plot
for node_name in group_nodes:
    plt.figure(figsize=(14, 4))

    for i, model in enumerate(loaded_mD2TbF_models):
        try:
            trace = model.nodes_db.node[node_name].trace()
            plt.plot(trace, label=f'Chain {i}', color=colors[i], alpha=0.7, lw=0.4)
        except Exception as e:
            print(f"Could not plot {node_name} for Chain {i}: {e}")

    # Aesthetics
    plt.title(f"Convergence Diagnostic: 8-Chain Trace Plot ({node_name})", fontsize=13, loc='left')
    plt.xlabel("Iteration (Post-Burn-in)", fontsize=11)
    plt.ylabel("Parameter Value", fontsize=11)

    plt.gca().spines['top'].set_visible(False)
    plt.gca().spines['right'].set_visible(False)
    plt.legend(loc='center left', bbox_to_anchor=(1, 0.5), frameon=False, ncol=1)

    plt.tight_layout()

    # --- SAVE LOGIC ---
    # Sanitize filename (remove characters like ':', '[', ']' which cause OS errors)
    safe_filename = node_name.replace(':', '_').replace('[', '').replace(']', '').replace('(', '').replace(')', '')
    save_path = os.path.join(output_folder, f"trace_{safe_filename}.png")

    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    # ------------------

    plt.show()
    plt.close() # Important to free up memory when saving many plots
```

```
print(f"Done! All group-level 'hairy caterpillars' are saved in '{output_folder}'.")
```

In [ ]:
```
Winning model - extract per-subject model estimates across DBS states
```

In [ ]:
```
import pandas as pd
import numpy as np

# ==============================================================================
# 1. EXTRACT AND COMBINE ALL TRACES
# ==============================================================================
print("Extracting posterior traces from all chains...")
# Assuming loaded_mD4_models is your list of Model 4 objects
traces = pd.concat([model.get_traces() for model in loaded_mD2TbF_models], ignore_index=True)

# ==============================================================================
# 2. EXACT COLUMN MAPPING (STN and GPi Group Nodes)
# ==============================================================================
nodes = {
    # Baseline Speed (Intercepts)
    'v_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]',
    'v_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]',
    'v_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]',
    'v_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]',
    'v_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]',
    'v_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]',

    # Effort Slopes
    'eff_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]:eff_z',
    'eff_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]:eff_z',
    'eff_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]:eff_z',
    'eff_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]:eff_z',
    'eff_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]:eff_z',
    'eff_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]:eff_z',

    # Reward Slopes
    'rew_STN_Pre': 'v_DBS_location[0_STN]:DBS_state[0_PreDBS]:rew_z',
    'rew_GPi_Pre': 'v_DBS_location[1_GPi]:DBS_state[0_PreDBS]:rew_z',
    'rew_STN_OFF': 'v_DBS_location[0_STN]:DBS_state[1_OFF]:rew_z',
    'rew_GPi_OFF': 'v_DBS_location[1_GPi]:DBS_state[1_OFF]:rew_z',
```

```
    'rew_STN_ON':  'v_DBS_location[0_STN]:DBS_state[2_ON]:rew_z',
    'rew_GPi_ON':  'v_DBS_location[1_GPi]:DBS_state[2_ON]:rew_z',


    # Non decision time
    't_STN_Pre': 't(0_STN.0_PreDBS)',
    't_GPi_Pre': 't(1_GPi.0_PreDBS)',
    't_STN_OFF': 't(0_STN.1_OFF)',
    't_GPi_OFF': 't(1_GPi.1_OFF)',
    't_STN_ON':  't(0_STN.2_ON)',
    't_GPi_ON':  't(1_GPi.2_ON)'
}

# ==============================================================================
# 3. SUBJECT-LEVEL EXTRACTION ENGINE
# ==============================================================================
print("Calculating subject-level posterior estimates...")

trace_means = traces.mean()

def map_to_label(col_name, nodes_dict):
    """Parses an HDDM subject node string and maps it back to our clean group labels."""
    if '.' not in col_name:
        return None, None

    parts = col_name.rsplit('.', 1)
    base_name = parts[0]
    subj_id = parts[1]

    clean_base = base_name.replace('_subj', '')

    for label, group_string in nodes_dict.items():
        if clean_base == group_string:
            return label, subj_id

    return None, None

data_records = []

for col_name, mean_val in trace_means.items():
    label, subj_id = map_to_label(col_name, nodes)
```

```
    if label is not None:
        data_records.append({
            'Subject': subj_id,
            'Parameter': label,
            'Estimate': mean_val
        })

df_long = pd.DataFrame(data_records)
df_wide = df_long.pivot(index='Subject', columns='Parameter', values='Estimate').reset_index()

# ==============================================================================
# 4. SCRUB GHOST PARAMETERS & ASSIGN GROUP
# ==============================================================================
print("Cleaning ghost parameters and assigning DBS_location...")

# Create masks based on the ground-truth threshold parameter
is_stn = df_wide['t_STN_Pre'].notna()
is_gpi = df_wide['t_GPi_Pre'].notna()

stn_cols = [col for col in df_wide.columns if '_STN_' in col]
gpi_cols = [col for col in df_wide.columns if '_GPi_' in col]

# Erase STN variables for GPi subjects, and vice versa
df_wide.loc[~is_stn, stn_cols] = np.nan
df_wide.loc[~is_gpi, gpi_cols] = np.nan

# Create the new grouping variable
df_wide['DBS_location'] = np.where(is_stn, 'STN', 'GPi')

# ==============================================================================
# 5. UNIFY COLUMNS (REMOVE STN/GPi FROM NAMES)
# ==============================================================================
print("Unifying columns across groups...")

params = ['v', 'eff', 'rew', 't']
states = ['Pre', 'OFF', 'ON']

for p in params:
    for s in states:
        stn_col = f'{p}_STN_{s}'
```

```
        gpi_col = f'{p}_GPi_{s}'
        unified_col = f'{p}_{s}'

        if stn_col in df_wide.columns and gpi_col in df_wide.columns:
            # Merge the valid STN and GPi numbers into a single column
            df_wide[unified_col] = df_wide[stn_col].fillna(df_wide[gpi_col])

            # Drop the old split columns to clean up the dataframe
            df_wide.drop(columns=[stn_col, gpi_col], inplace=True)

# ==============================================================================
# 6. CALCULATE SHIFTS ON UNIFIED COLUMNS
# ==============================================================================
print("Calculating within-subject shifts (OFF minus PreDBS) and (ON minus OFF)...")

for p in params:
    pre_col = f'{p}_Pre'
    off_col = f'{p}_OFF'
    on_col = f'{p}_ON'

    # Shift 1: OFF minus PreDBS
    if pre_col in df_wide.columns and off_col in df_wide.columns:
        df_wide[f'{p}_Shift_OFF_minus_Pre'] = df_wide[off_col] - df_wide[pre_col]

    # Shift 2: ON minus OFF
    if off_col in df_wide.columns and on_col in df_wide.columns:
        df_wide[f'{p}_Shift_ON_minus_OFF'] = df_wide[on_col] - df_wide[off_col]

# Reorder the columns so 'Subject' and 'DBS_location' are up front
cols = ['Subject', 'DBS_location'] + [c for c in df_wide.columns if c not in ['Subject', 'DBS_location']]
df_wide = df_wide[cols]

# ==============================================================================
# 7. EXPORT TO CSV
# ==============================================================================
export_filename = "HDDM_SubjectLevel_Estimates_Unified_mD2TbF_29May2026.csv"
df_wide.to_csv(export_filename, index=False)
print(f"\nSuccess! Unified subject-level data exported to '{export_filename}'")

# Display the first few rows to confirm structure in the notebook
try:
```

```
    display(df_wide.head())
except NameError:
    print(df_wide.head())
```

In [ ]: